



Linux Fundamentals

0%



Section 17 / 30

[Go to Questions](#) ↓

Service and Process Management

Services, also known as daemons, are fundamental components of a Linux system that run silently in the background "without direct user interaction". They perform crucial tasks that keep the system operational and provide additional functionalities. Generally, services can be categorized into two types:

System Services

These are internal services required during system startup. They perform essential hardware-related tasks and initialize system components necessary for the operating system to function properly. These are like the engine and transmission systems. They start when you turn the ignition key and are essential for the car to run. Without them, the car wouldn't move.

User-Installed Services

These services are added by users and typically include server applications and other background processes that provide specific features or capabilities. These types of services are like the car's air conditioning or GPS navigation system. While not critical for the car to operate, they enhance functionality and provide additional features based on the driver's preferences.

Daemons are often identified by the letter **d** at the end of their program names, such as **sshd** (SSH daemon) or **systemd**. Just as a car relies on both its core components and optional features to provide a complete experience, a Linux system utilizes both system and user-installed services to function efficiently and meet user needs.

In general, there are just a few goals that we have when we deal with a service or a process:

1. Start/Restart a service/process
2. Stop a service/process
3. See what is/was happening with a service/process
4. Enable/Disable a service/process on boot
5. Find a service/process

Most modern Linux distributions have adopted **systemd** as their initialization system (init system). It is the first process that starts during the boot process and is assigned the Process ID (**PID**). All processes in a Linux system are assigned a **PID** and can be viewed under the **/proc/** directory, which contains information about each process. Processes may also have a Parent Process ID (**PPID**), indicating that they were started by another process (the parent), making them child processes.

Systemctl

After installing **OpenSSH** on our VM, we can start the service with the following command.

```
athane@htb[/htb]$ systemctl start ssh
```

shellsession

After we have started the service, we can now check if it runs without errors.

shellsession

```
athane@htb[/htb]$ systemctl status ssh
```

- ssh.service - OpenBSD Secure Shell server
Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
Active: active (running) since Thu 2020-05-14 15:08:23 CEST; 24h ago
Main PID: 846 (sshd)
Tasks: 1 (limit: 4681)
CGroup: /system.slice/ssh.service
└─846 /usr/sbin/sshd -D

```
Mai 14 15:08:22 inlane systemd[1]: Starting OpenBSD Secure Shell server...
Mai 14 15:08:23 inlane sshd[846]: Server listening on 0.0.0.0 port 22.
Mai 14 15:08:23 inlane sshd[846]: Server listening on :: port 22.
Mai 14 15:08:23 inlane systemd[1]: Started OpenBSD Secure Shell server.
Mai 14 15:08:30 inlane systemd[1]: Reloading OpenBSD Secure Shell server.
Mai 14 15:08:31 inlane sshd[846]: Received SIGHUP; restarting.
Mai 14 15:08:31 inlane sshd[846]: Server listening on 0.0.0.0 port 22.
Mai 14 15:08:31 inlane sshd[846]: Server listening on :: port 22.
```

To add OpenSSH to the SysV script to tell the system to run this service after startup, we can link it with the following command:

shellsession

```
athane@htb[/htb]$ systemctl enable ssh
```

Synchronizing state of ssh.service with SysV service script with
/lib/systemd/systemd-sysv-install.

Executing: /lib/systemd/systemd-sysv-install enable ssh

Once we reboot the system, the OpenSSH server will automatically run. We can check this with a tool called `ps`.

```
athane@htb[/htb]$ ps -aux | grep ssh
```

```
root      846  0.0  0.1  72300  5660 ?        Ss   May14   0:00 /usr/sbin/sshd -D
```

We can also use `systemctl` to list all services.

```
athane@htb[/htb]$ systemctl list-units --type=service
```

UNIT	LOAD	ACTIVE	SUB
accounts-daemon.service	loaded	active	running
Accounts Service			
acpid.service	loaded	active	running ACPI
event daemon			
apache2.service	loaded	active	running The
Apache HTTP Server			
apparmor.service	loaded	active	exited

```
AppArmor initialization
apport.service                loaded active exited LSB:
automatic crash repor
avahi-daemon.service          loaded active running
Avahi mDNS/DNS-SD Stack
bolt.service                  loaded active running
Thunderbolt system service
```

It is quite possible that the services do not start due to an error. To see the problem, we can use the tool `journalctl` to view the logs.

```
shellsession
athane@htb[/htb]$ journalctl -u ssh.service --no-pager

-- Logs begin at Wed 2020-05-13 17:30:52 CEST, end at Fri 2020-05-15 16:00:14 CEST. -
-
Mai 13 20:38:44 inlane systemd[1]: Starting OpenBSD Secure Shell server...
Mai 13 20:38:44 inlane sshd[2722]: Server listening on 0.0.0.0 port 22.
Mai 13 20:38:44 inlane sshd[2722]: Server listening on :: port 22.
Mai 13 20:38:44 inlane systemd[1]: Started OpenBSD Secure Shell server.
Mai 13 20:39:06 inlane sshd[3939]: Connection closed by 10.22.2.1 port 36444
[preauth]
Mai 13 20:39:27 inlane sshd[3942]: Accepted password for master from 10.22.2.1 port
36452 ssh2
Mai 13 20:39:27 inlane sshd[3942]: pam_unix(sshd:session): session opened for user
master by (uid=0)
Mai 13 20:39:28 inlane sshd[3942]: pam_unix(sshd:session): session closed for user
master
Mai 14 02:04:49 inlane sshd[2722]: Received signal 15; terminating.
```

```
Mai 14 02:04:49 inline systemd[1]: Stopping OpenBSD Secure Shell server...
Mai 14 02:04:49 inline systemd[1]: Stopped OpenBSD Secure Shell server.
-- Reboot --
```

Kill a Process

A process can be in the following states:

- Running
- Waiting (waiting for an event or system resource)
- Stopped
- Zombie (stopped but still has an entry in the process table).

Processes can be controlled using `kill`, `pkill`, `pgrep`, and `killall`. To interact with a process, we must send a signal to it. We can view all signals with the following command:

```
athane@htb[/htb]$ kill -l
```

shellsession

1) SIGHUP	2) SIGINT	3) SIGQUIT	4) SIGILL	5) SIGTRAP
6) SIGABRT	7) SIGBUS	8) SIGFPE	9) SIGKILL	10) SIGUSR1
11) SIGSEGV	12) SIGUSR2	13) SIGPIPE	14) SIGALRM	15) SIGTERM
16) SIGSTKFLT	17) SIGCHLD	18) SIGCONT	19) SIGSTOP	20) SIGTSTP
21) SIGTTIN	22) SIGTTOU	23) SIGURG	24) SIGXCPU	25) SIGXFSZ
26) SIGVTALRM	27) SIGPROF	28) SIGWINCH	29) SIGIO	30) SIGPWR
31) SIGSYS	34) SIGRTMIN	35) SIGRTMIN+1	36) SIGRTMIN+2	37) SIGRTMIN+3
38) SIGRTMIN+4	39) SIGRTMIN+5	40) SIGRTMIN+6	41) SIGRTMIN+7	42) SIGRTMIN+8

```
43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7
58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3  62) SIGRTMAX-2
63) SIGRTMAX-1  64) SIGRTMAX
```

The most commonly used signals are:

Signal	Description
1	<code>SIGHUP</code> - This is sent to a process when the terminal that controls it is closed.
2	<code>SIGINT</code> - Sent when a user presses <code>[Ctrl] + C</code> in the controlling terminal to interrupt a process.
3	<code>SIGQUIT</code> - Sent when a user presses <code>[Ctrl] + D</code> to quit.
9	<code>SIGKILL</code> - Immediately kill a process with no clean-up operations.
15	<code>SIGTERM</code> - Program termination.
19	<code>SIGSTOP</code> - Stop the program. It cannot be handled anymore.
20	<code>SIGTSTP</code> - Sent when a user presses <code>[Ctrl] + Z</code> to request for a service to suspend. The user can handle it afterward.

For example, if a program were to freeze, we could force to kill it with the following command:

shellsession

```
athane@htb[/htb]$ kill 9 <PID>
```

Background a Process

Sometimes it will be necessary to put the scan or process we just started in the background to continue using the current session to interact with the system or start other processes. As we have already seen, we can do this with the shortcut **[Ctrl + Z]**. As mentioned above, we send the **SIGTSTP** signal to the kernel, which suspends the process.

shellsession

```
athane@htb[/htb]$ ping -c 10 www.hackthebox.eu
```

```
PING www.hackthebox.eu (104.20.55.68) 56(84) bytes of data.
```

```
[Ctrl + Z]
```

```
[1]+  Stopped                  ping -c 10 www.hackthebox.eu
```

```
athane@htb[/htb]$ vim tmpfile
```

```
[Ctrl + Z]
```

```
[2]+  Stopped                  vim tmpfile
```

Now all background processes can be displayed with the following command.

shellsession

```
athane@htb[/htb]$ jobs
```



```
[1]+  Stopped                  ping -c 10 www.hackthebox.eu
[2]+  Stopped                  vim tmpfile
```

The `[Ctrl] + Z` shortcut suspends the processes, and they will not be executed further. To keep it running in the background, we have to enter the command `bg` to put the process in the background.

```
athane@htb[/htb]$ bg
```

```
[1]+  ping -c 10 www.hackthebox.eu &
```

```
athane@htb[/htb]$
```

```
--- www.hackthebox.eu ping statistics ---
```

```
10 packets transmitted, 0 received, 100% packet loss, time 113482ms
```

```
[ENTER]
```

```
[1]+  Exit 1                  ping -c 10 www.hackthebox.eu
```

shellsession

Another option is to automatically set the process with an AND sign (`&`) at the end of the command.

```
athane@htb[/htb]$ ping -c 10 www.hackthebox.eu &
```

```
[1] 10825
```

```
PING www.hackthebox.eu (172.67.1.1) 56(84) bytes of data.
```

shellsession

Once the process finishes, we will see the results.

```
shellsession  
  
athane@htb[/htb]$  
  
--- www.hackthebox.eu ping statistics ---  
10 packets transmitted, 0 received, 100% packet loss, time 9210ms  
  
[ENTER]  
[1]+  Exit 1                  ping -c 10 www.hackthebox.eu
```

Foreground a Process

After that, we can use the `jobs` command to list all background processes. Backgrounded processes do not require user interaction, and we can use the same shell session without waiting until the process finishes first. Once the scan or process finishes its work, we will get notified by the terminal that the process is finished.

```
shellsession  
  
athane@htb[/htb]$ jobs  
  
[1]+  Running                  ping -c 10 www.hackthebox.eu &
```

If we want to get the background process into the foreground and interact with it again, we can use the `fg <ID>` command.

```
athane@htb[/htb]$ fg 1
ping -c 10 www.hackthebox.eu

--- www.hackthebox.eu ping statistics ---
10 packets transmitted, 0 received, 100% packet loss, time 9206ms
```

shellsession

Execute Multiple Commands

There are three possibilities to run several commands, one after the other. These are separated by:

- Semicolon (`;`)
- Double ampersand characters (`&&`)
- Pipes (`|`)

The difference between them lies in the previous processes' treatment and depends on whether the previous process was completed successfully or with errors. The semicolon (`;`) is a command separator and executes the commands by ignoring previous commands' results and errors.

shellsession

```
athane@htb[/htb]$ echo '1'; echo '2'; echo '3'
```

```
1
2
3
```

For example, if we execute the same command but replace it in second place, the command `ls` with a file that does not exist, we get an error, and the third command will be executed nevertheless.

```
athane@htb[/htb]$ echo '1'; ls MISSING_FILE; echo '3'
```

```
1
ls: cannot access 'MISSING_FILE': No such file or directory
3
```

However, it looks different if we use the double AND characters (`&&`) to run the commands one after the other. If there is an error in one of the commands, the following ones will not be executed anymore, and the whole process will be stopped.

```
athane@htb[/htb]$ echo '1' && ls MISSING_FILE && echo '3'
```

```
1
ls: cannot access 'MISSING_FILE': No such file or directory
```

Pipes (|) depend not only on the correct and error-free operation of the previous processes but also on the previous processes' results.

Connect to HTB

Target(s)

Spawn the target system to get IPs and answer questions

 Spawn the target system



Enable step-by-step solutions  PRO

Question 1

 +1

 +20 ^

Use the "systemctl" command to list all units of services and submit the unit name with the description "Load AppArmor profiles managed internally by snapd" as the answer.

SSH to with user "**htb-student**" and password "**HTB_@cademy_stdnt!**"

Write your answer

 Submit

 Hint



17 / 30 Sections



Mark Complete & Next